

ECOTE – Final Project

Alejandro Rodríguez González

13th May 2026

Semester: Summer

Subject: Write a Bottom-Up parser with backtracking. Output also the phases of the syntax tree development.

1 General overview and assumptions

The aim of this project is to implement a bottom-up parser with backtracking as part of the syntax analysis phase of a compiler. A parser checks whether an input string belongs to the language generated by a grammar. If the string is valid, the parser produces a parse tree; otherwise, it rejects the string.

The parser follows a bottom-up strategy. It starts from the input symbols and progressively builds the parse tree upwards towards the start symbol. In practical terms, the parser uses shift-reduce parsing:

- it shifts input symbols,
- reduces sequences that match the right-hand side of grammar productions,
- applies backtracking when a chosen reduction path does not lead to a valid parse tree.

When detailed output is enabled, the implementation also outputs the phases of the syntax tree construction. Each relevant phase corresponds to the parser state after a shift, reduction, or backtracking step. This makes it possible to follow the evolution of the parsing process step by step.

The assumptions of the project are:

- The input is already a valid sequence of lexical units produced by a previous scanning stage.
- Lexical analysis is outside the scope of this project.
- The grammar is context-free.
- The grammar is assumed not to contain cycles. However, the implementation explicitly detects unit cycles.
- The grammar does not contain ε -productions.

2 Functional requirements

2.1 FR1. Grammar input

The system has to accept as input a context-free grammar

$$G = \langle T, N, P, S \rangle$$

where:

- T is the set of terminals.
- N is the set of nonterminals.
- P is the set of productions.
- S is the start symbol.

2.2 FR2. Grammar file

The grammar must be read entirely from the file given as the first command-line argument. A valid grammar file has the following form:

```
START S
S -> S A / a
A -> A b S / b
```

In this format:

- the first line declares the start symbol;
- each following line defines one grammar rule;
- the left-hand side contains one uppercase nonterminal symbol;
- the right-hand side may contain one or more alternatives separated by $/$.

The following grammar defines the format of this grammar file:

$$\{T_M, N_M, P_M, GrammarFile\}$$

where:

$$T_M = \{START, \rightarrow, /, EOL, blank, UppercaseLetter, LowercaseLetter\}$$

and

$$N_M = \{GrammarFile, StartLine, ProductionList, AlternativeList, RightHandSide, Symbol, Nonterminal, Terminal, WS\}.$$

```
GrammarFile -> StartLine EOL ProductionList
```

```
StartLine -> START WS Nonterminal
```

```
ProductionList -> ProductionLine
                | ProductionLine EOL ProductionList
```

```
ProductionLine -> Nonterminal WS -> WS AlternativeList
```

```
AlternativeList -> RightHandSide
                | RightHandSide WS / WS AlternativeList
```

```
RightHandSide -> Symbol
                | Symbol WS RightHandSide
```

```
Symbol -> Nonterminal
        | Terminal
```

```
Nonterminal -> UppercaseLetter
```

Terminal -> LowercaseLetter

WS -> blank
| blank WS

2.3 FR3. Grammar file error detection

For malformed grammar lines the system shall be capable of detecting the line on which a grammar error occurs and of reporting both what was expected and what was found. For example:

ERROR: Grammar error at line 2: expected non-empty alternative after '/',
found 'S -> a /'

The grammar-error detection system shall detect at least:

- An empty grammar file.
- A missing or invalid **START** declaration.
- A grammar without productions after the **START** declaration.
- Malformed production lines.
- Missing symbols after ->.
- Empty alternatives after /.
- Use of ε -productions.
- A start symbol that does not appear on the left-hand side of any production.
- Invalid left-hand sides containing more than one symbol or more than one character.
- Invalid left-hand sides whose symbol is not uppercase, such as **s** -> **a**.
- Unit cycles such as **S** -> **S**.

For example, if a production has a lowercase symbol on the left-hand side, the system reports:

ERROR: Grammar error at line 2: expected one uppercase nonterminal on
the left side, found 's'

2.4 FR4. Program parameters

The system shall accept three required input parameters:

1. the path to the grammar file;
2. the path to the words file;
3. the path to the output file.

Optionally, the system may accept a fourth parameter indicating whether the detailed parsing trace has to be printed.

Therefore the execution is:

```
python main.py grammar.txt words.txt output.txt [detailed_output: bool]
```

2.5 FR5. Words file

The system has to accept as input a words file containing words composed of terminal symbols of the grammar. The words file shall store multiple words separated by the symbol \$, which marks the end of each word. The words file has to satisfy the following conditions:

- it must not be empty;
- it must end with the final \$ marker;
- it must not contain empty words;
- it must be readable from the path passed as the second command-line argument.

2.6 FR6. Words file error detection

The system shall report malformed or unreadable words files using descriptive error messages. For example:

```
ERROR: Words file must end with $.
```

```
ERROR: Empty words are not supported.
```

```
ERROR: Unable to read words file: words.txt (No such file or directory)
```

2.7 FR7. Output file

The system shall accept as input the path to an output file in which the program output will be written. This file shall store both the normal parsing results and any error messages produced during execution after the output file has been opened.

2.8 FR8. Grammar restrictions

Since the project requires a bottom-up parser with backtracking, the grammar has to satisfy the following restrictions:

- the grammar should contain no cycles; in the current implementation, unit cycles are explicitly detected;
- the grammar should contain no ε -productions.

2.9 FR9. Parsing objective

The parser should attempt to reconstruct, in reverse, a rightmost derivation of each input string read from the words file. For every string, the parser shall therefore operate according to the bottom-up strategy, starting from the leaves of the future parse tree and progressing towards the root.

2.10 FR10. Shift-reduce behaviour

The parser has to implement shift-reduce parsing. During parsing:

- if a substring matches the right-hand side of a production, it may be reduced to the symbol on the left-hand side of that production;
- if no reduction is possible and unread input symbols still remain, the parser has to shift by advancing the input pointer.

2.11 FR11. Reduction rule

At each reduction step, the parser has to identify a substring matching the right-hand side of a production and replace it with the corresponding left-hand side nonterminal. The parser has to consider all possible reductions whenever necessary.

2.12 FR12. Backtracking behaviour

If all input symbols have already been read and a complete parse tree has not been obtained, the parser has to perform backtracking. Backtracking should allow the parser to:

- remove one production, that is, one level of the current parse tree;
- reset the input pointer when needed;
- try alternative reductions.

If all alternatives have been checked and no valid parse tree can be obtained for a given input string read from the words file, that input string has to be rejected.

2.13 FR13. Parse tree construction

If a given input string from the words file is accepted, the system has to construct a parse tree representing the syntactic structure of that string. The leaves of the tree, read from left to right, have to correspond to the accepted input string.

2.14 FR14. Output of syntax tree development phases

At a minimum, when detailed output is enabled, the output shall show the intermediate states produced after:

- each shift;
- each reduction;
- each backtracking step;
- the final acceptance or rejection of each input string.

If detailed output is not enabled, the output shall only show the final acceptance or rejection of each input string, the parse tree and its reversed rightmost derivation.

2.15 FR15. Acceptance condition

The system has to accept each input string from the words file if it is possible to obtain a complete parse tree whose root is the start symbol of the grammar and whose leaves match that input string.

2.16 FR16. Rejection condition

The system has to reject an input string from the words file if that string is not generated by the grammar, that is, if all parsing and backtracking alternatives have been exhausted without obtaining a valid parse tree for that string.

2.17 FR17. Scope limitation

The system shall perform syntax analysis only. Lexical analysis, semantic analysis, code optimisation, and code generation are outside the scope of this project. The parser may assume that the input is already provided as grammar symbols or tokens.

3 Implementation

3.1 General architecture

This project separates grammar loading, parser control, parse-tree construction, output formatting, output writing, and diagnostics into distinct components, in particular:

- **Input Reader:** reads the grammar specification, the words from a separate file, and the output path.
- **Grammar Manager:** validates the grammar and builds the internal grammar object.
- **Parser Engine:** controls the parsing loop and performs shift, reduce, and backtrack operations.
- **Forest Builder:** maintains the current partial parse forest and updates it after every accepted shift and reduction.
- **Output Formatter:** prints each parsing phase in the required textual layout.
- **Output Writer:** writes normal output and error messages to the output file.
- **Error Handler:** reports invalid grammar format, malformed input, and unsuccessful parsing attempts.

At runtime, the control flow is as follows:

1. The output file is opened if the correct number of command-line parameters is provided.
2. The grammar is read.
3. The words file is read and validated.
 - 3.1 The complete file content is read.
 - 3.2 Newline characters are removed.
 - 3.3 The content is checked to ensure that it is not empty.
 - 3.4 The final \$ marker is checked.
 - 3.5 The content is split into words using \$.
 - 3.6 Empty words are rejected.
4. The grammar is validated.
 - 4.1 Empty lines are ignored and the remaining lines are stored together with their original line number.
 - 4.2 The first non-empty line is checked as the **START** declaration. If it is not valid, the error reports the line, the expected format, and the found text.
 - 4.3 The program checks that there is at least one production after the **START** declaration.
 - 4.4 Each production line is checked for the production arrow $\rightarrow$.
 - 4.5 The left-hand side is checked. It must contain exactly one uppercase nonterminal.
 - 4.6 The right-hand side is checked. It must contain symbols after $\rightarrow$.
 - 4.7 The start symbol is checked. It must appear on the left-hand side of at least one production.
 - 4.8 The right-hand side alternatives are parsed. Empty alternatives after / are rejected.
 - 4.9 The production symbols are checked. ε -productions are rejected.

- 4.10 Unit cycles are detected after all productions have been created.
5. For each word, the initial forest of terminal leaves is created, and the parser repeatedly searches for reducible substrings, applies one reduction, logs the new state, and continues.
- 5.1 If no reduction is possible and unread input still remains, the parser performs a shift.
- 5.2 If the whole input has been read and the parse tree is still incomplete, the parser restores a previous state and tries another reduction path.
6. Parsing for a given word finishes either when a complete parse tree rooted at the start symbol is obtained or when all alternatives have been exhausted.

3.2 Data structures

The following conceptual data structures are used:

3.2.1 GrammarLine

- `number: int`
- `text: str`

The `GrammarLine` structure stores one non-empty grammar line together with its original line number. It is used by the grammar-error detection system to report where an error was found.

3.2.2 Production

- `id: int`
- `left_side: str`
- `right_side: list[str]`
- `source_line: int | None`

The `source_line` field stores the grammar line from which the production was created. It is especially useful when reporting production-related errors, such as unit cycles.

3.2.3 Grammar

- `terminals: set[str]`
- `nonterminals: set[str]`
- `start_symbol: str`
- `productions: list[Production]`
- `by_left_side: dict[str, list[Production]]`

The parse tree is represented as a forest of nodes. A single node may be either a terminal leaf or a nonterminal node produced by a reduction. Since the frontier is defined by the leaves of the tree read from left to right, the current forest is sufficient both to build the final parse tree and to compute the current frontier shown after every step.

3.2.4 Node

- `symbol: str`
- `production_id: int | None`
- `children: list[Node]`

The parser state must support backtracking. For that reason, the implementation can clone the current forest, the consumed nodes, the remaining input, and the derivation history. In Python, backtracking is implemented recursively, so the call stack stores the active search path.

3.2.5 ParserState

- `input_pos: int`
- `forest: list[Node]`
- `consumed: list[Node]`
- `remaining_input: list[str]`
- `right_most_derivation_hist: list[int]`

3.2.6 BacktrackFrame

- `state: ParserState`
- `alternatives: list[Match]`

In the current implementation, backtracking is mainly driven by recursive calls. The `BacktrackFrame` stores a cloned state and a list of alternative matches. In the current implementation, recursive calls drive the backtracking process.

3.2.7 Match

- `production: Production`
- `start_index: int`
- `end_index: int`

Finally, every visible phase is stored in a step record so that the output formatter can print it in a text-based tree layout.

3.2.8 StepRecord

- `step_number: int`
- `action: str`
- `symbol: str | None`
- `production: str | None`
- `undo: str | None`
- `working_part: str`
- `frontier: str`
- `forest_snapshot: str`

3.3 Module descriptions

3.3.1 Input Reader

The Input Reader is responsible for reading all external input required by the program. In this implementation, the grammar is loaded from a text file passed as the first command-line argument, while the words are loaded from the words file passed as the second command-line argument. The module:

1. Validates that all required arguments are present.
2. Reads the grammar file.
3. Reads the words file.
4. Verifies the word delimiters.
5. Stores the output file path.

```
read_input(argv):
    if the number of command-line arguments is not between 4 and 5:
        raise "Expected at least 3 CLI parameters."

    grammar_path = argv[1]
    words_path = argv[2]
    output_path = argv[3]

    detailed_output = True
    if a fifth argument exists:
        detailed_output = argument equals "True" or "true"

    grammar_text = read_grammar_file(grammar_path)
    input_words = read_words_file(words_path)

    store grammar_text
    store output_path
    store input_words
    store detailed_output
```

The words file is processed by a specific routine:

```
read_words_file(words_path):
    try:
        open words_path
        read complete file content
        remove newline and carriage return characters

        if content is empty:
            raise "Words file is empty."

        if content does not end with '$':
            raise "Words file must end with $."

    words = split content by '$', excluding the last empty part
```

```

    if any word is empty:
        raise "Empty words are not supported."

    return each word with '$' restored at the end

if the file cannot be read:
    raise "Unable to read words file: path (reason)"

```

If the words file cannot be opened, the module raises an error indicating the path and the operating-system reason.

3.3.2 Grammar Manager

The Grammar Manager:

- Converts the raw grammar text into the internal grammar representation.
- It extracts the start declaration, the left-hand side nonterminals, and the right-hand side alternatives
- It computes the sets of terminals and nonterminals.
- It detects all grammar-related errors defined for the input format and reports the original source line together with a message stating what was expected and what was found.

Cycle detection has a deliberate limitation in the current implementation. The Grammar Manager explicitly detects only unit cycles, that is, cycles formed exclusively by unit productions. For example:

S -> S

or:

S -> A

A -> S

```

parse_grammar(grammar_text):
    lines = collect_grammar_lines(grammar_text)

    if lines is empty:
        raise "Grammar file is empty."

    start_line = lines[0]
    start_symbol = parse_start_declaration(start_line)

    production_lines = all lines after start_line

    if production_lines is empty:
        raise grammar error after the START declaration

    return build_grammar(start_line, start_symbol, production_lines)

build_grammar(start_line, start_symbol, production_lines):
    nonterminals = get_nonterminals(production_lines)

    if start_symbol is not in nonterminals:

```

```

        raise grammar error at start_line

productions = parse_productions(production_lines)

grammar = Grammar(
    terminals = infer_terminals(nonterminals, productions),
    nonterminals = nonterminals,
    productions = productions,
    start_symbol = start_symbol,
    by_left_side = group_by_left_side(productions)
)

validate that grammar has no unit cycles

return grammar

parse_productions(production_lines):
    productions = empty list
    production_id = 1

    for each line in production_lines:
        left_side, arrow, right_side = split_production(line)
        alternatives = split right_side by '/'

        for each alternative:
            remove surrounding blanks

            if alternative is empty:
                raise grammar error at line

            symbols = split alternative by blanks

            if symbols is empty or contains ' $\epsilon$ ':
                raise grammar error at line

            create Production(
                id = production_id,
                left_side = left_side,
                right_side = symbols,
                source_line = line number
            )

            add production to productions
            production_id = production_id + 1

    return productions

```

This module uses `GrammarLine` to preserve original line numbers and `Production` to store each internal production together with the line where it was declared.

3.3.3 Parser Engine

The Parser Engine is the central module of the application. It receives the input symbols of one word, searches recursively for a valid parse state, and returns either an accepted or rejected parse result.

Within this component, two internal procedures are especially important:

- a reduction-matching procedure, which scans the current forest and finds all substrings matching the right-hand side of a production;
- a backtracking procedure, which restores a previously saved parser state and tries the next available reduction alternative.

```
parse(input_symbols):
    reset step counter
    store a copy of input_symbols

    build full forest in the initial parser state
    initial_state = clone current parser state

    final_state = search(initial_state)

    if final_state is None:
        return rejected ParseResult with initial_state

    final_root = clone the only consumed node in final_state
    return accepted ParseResult with final_state and final_root
```

Internal reduction-matching procedure:

```
find_matches(state):
    matches = empty list
    working_symbols = symbols from state.consumed

    for each start_index in the indexes of working_symbols:
        for each production in grammar productions:
            end_index = start_index + length of production right side

            if end_index is outside consumed nodes:
                continue

            if working_symbols[start_index:end_index]
               equals production.right_side:
                add Match(production, start_index, end_index)

    return matches
```

Internal backtracking function:

```
search(state):
    if state is accepted:
        return state

    matches = find_matches(state)
```

```

if matches is not empty:
    base_state = clone state
    frame_state = clone base_state

    for each match in matches:
        candidate_state = clone frame_state

        apply reduction to candidate_state
        record REDUCE step

        result = search(candidate_state)

        if result is not None:
            return result

        restored_state = clone base_state
        record BACKTRACK step for the undone production

    if state still has remaining input:
        shifted_state = clone base_state
        symbol = first remaining input symbol

        apply shift to shifted_state
        record SHIFT step

        return search(shifted_state)

    if state has remaining input:
        shifted_state = clone state
        symbol = first remaining input symbol

        apply shift to shifted_state
        record SHIFT step

        return search(shifted_state)

return None

```

This component is the one that determines whether each input word is accepted or rejected. If parsing succeeds, it also produces the final parse tree and the sequence of productions used in the reversed rightmost derivation.

3.3.4 Forest Builder

The Forest Builder maintains the current partial parse forest throughout execution. A shift operation creates a new terminal leaf, appends it to the consumed part, removes the symbol from the remaining input, and rebuilds the visible forest. A reduction operation replaces a contiguous subsequence of nodes by a new nonterminal node whose children are exactly the matched nodes.

```

apply_shift(symbol):
    create terminal node from symbol

```

```

append terminal node to consumed nodes

if there is remaining input:
    remove the first remaining input symbol

increase input position
rebuild full forest

apply_reduction(match):
    nodes_involved = consumed[match.start_index:match.end_index]

    reduced = create reduction node using:
        production = match.production
        children = nodes_involved

    replace consumed slice with reduced node
    append production id to reversed rightmost derivation history
    rebuild full forest

```

3.3.5 Output Formatter

The Output Formatter generates the required text-based output. For each parsing step, it formats the step number, the action type, the relevant symbol or production, the current working part, the remaining frontier, and the current forest.

```

record_step(step_number, action, state, symbol, production, undo):
    create StepRecord with:
        step number
        action
        shifted symbol, if any
        reduced production, if any
        undone production, if any
        working part from consumed nodes
        frontier from remaining input
        rendered forest

    return formatted StepRecord

format_result(result):
    sections = ACCEPTED if result is accepted,
               otherwise REJECTED

    if result is accepted and has final root:
        add "FINAL PARSE TREE"
        add rendered final tree
        add "REVERSED RIGHTMOST DERIVATION"
        add formatted derivation history

    return sections joined by blank lines

```

3.3.6 Output Writer

The Output Writer is responsible for managing the output file. It opens the file in write mode, writes each formatted line, prints the same text to the standard output, and closes the file when

execution finishes.

```
write(text):
    print text to standard output
    write text to the output file

close():
    close the output file
```

3.3.7 Error Handler

The Error Handler reports grammar-format errors, missing command-line arguments, invalid input-word format, and invalid execution states. Rejected input words are formatted as parser results, not as error messages.

```
raise_error(message):
    raise ParserError(message)

format_error(error):
    return "ERROR: " + error message
```

3.4 Input and output description

The program is executed from the command line. The details of the required input parameters are specified in FR4. Program parameters. No interactive input is required during execution.

The grammar file contains the start symbol and the grammar productions. Production identifiers are not written in the input file; they are assigned internally by the Grammar Manager after reading the grammar.

The second command-line argument is a words file. It contains one or more input words composed of grammar terminals, separated by the marker \$. For example:

```
abba$baab$
```

The third command-line argument is the output file. The program writes its normal output and any error messages there in plain text. When detailed output is enabled, the generated output consists of two parts:

1. a step-by-step parsing trace;
2. the final result.

When detailed output is disabled, only the final result is written.

The step-by-step trace shows the development of the syntax tree during parsing. Each step contains:

- the step number;
- the parsing action;
- the symbol, production, or undone production involved;
- the current working part;
- the current frontier;
- the current forest.

The possible actions are:

- SHIFT
- REDUCE
- BACKTRACK

The trace is printed in the following style:

```
STEP 1
ACTION : SHIFT
SYMBOL : a
WORKING PART : a
FRONTIER : bba
FOREST
[0] a
[1] b
[2] b
[3] a

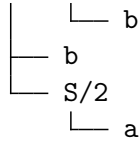
STEP 2
ACTION : REDUCE
PRODUCTION: 2) S -> a
WORKING PART : S
FRONTIER : bba
FOREST
[0] S/2
    └─ a
[1] b
[2] b
[3] a

STEP 3
ACTION : BACKTRACK
UNDO : 2) S -> a
WORKING PART : a
FRONTIER : bba
FOREST
[0] a
[1] b
[2] b
[3] a
```

If parsing succeeds, the final output includes the complete parse tree in a box-drawing text form:

FINAL PARSE TREE

```
S/1
├── S/2
│   └── a
└── A/3
    └── A/4
```



REVERSED RIGHTMOST DERIVATION

(2,4,2,3,1)

The program also prints the reversed rightmost derivation as the sequence of internally assigned production numbers used during parsing.

If parsing fails, the program prints a rejection message instead of a final parse tree, for example:

REJECTED

If one of the input sources is malformed, the program prints a descriptive error message. These errors may originate either in the *Grammar File* or in the *Words file*. Grammar-related errors are reported with detailed diagnostics, including the line number and the specific condition that failed.

- **Grammar File errors:** missing grammar file; missing or invalid **START** declaration; malformed production lines; missing symbols after $\rightarrow$; empty alternatives after $/$; use of ε -productions; invalid lowercase left-hand sides such as $s \rightarrow a$; unit cycles such as $S \rightarrow S$; a start symbol that does not appear on the left-hand side of any production; and invalid left-hand sides containing more than one nonterminal.
- **Words file errors:** missing words file; empty words, which are not supported; a words file that cannot be opened; and a words file that does not end with the final $\$$ marker.

4 Functional test cases

This project includes a functional test suite, which can be executed with the following command.

```
python3 tests/test.py
```

The suite runs a total of 20 functional tests and compares the obtained results with the expected outcomes.

ID	Grammar File	Words file	Expected result
#1	START S S $\rightarrow$ S A / a A $\rightarrow$ A b S / b	a\$	ACCEPTED
#2	START S S $\rightarrow$ S A / a A $\rightarrow$ A b S / b	ab\$	ACCEPTED
#3	START S S $\rightarrow$ S A / a A $\rightarrow$ A b S / b	abb\$	ACCEPTED
#4	START S S $\rightarrow$ S A / a A $\rightarrow$ A b S / b	abba\$	ACCEPTED

#5	START S S -> S A / a A -> A b S / b	aa\$	REJECTED
#6	START S S -> S A / a A -> A b S / b	b\$	REJECTED
#7	START S S -> S A / a A -> A b S / b	\$	ERROR: Empty words are not supported.
#8	START S S -> S A / a A -> A b S / b	abba	ERROR: Words file must end with \$.
#9	S -> S A / a A -> A b S / b	a\$	ERROR: Grammar error at line 1: expected START declaration in format 'START <symbol>', found 'S -> S A / a'
#10	START S S-SA/a A -> A b S / b	a\$	ERROR: Grammar error at line 2: expected production in format '<NonTerminal> -> <symbols>', found 'S-SA/a'
#11	START S S ->	a\$	ERROR: Grammar error at line 2: expected symbols after '->', found 'S ->'
#12	START S S -> S	a\$	ERROR: Grammar error at line 2: expected grammar without unit cycles, found unit cycle S -> S.
#13	START S S -> a / ϵ	a\$	ERROR: Grammar error at line 2: expected non-empty symbols without epsilon, found ' ϵ '
#14	START Z S -> a	a\$	ERROR: Grammar error at line 1: expected START symbol to appear on the left side of a production, found 'Z'
#15	START S S -> a /	a\$	ERROR: Grammar error at line 2: expected non-empty alternative after '/', found 'S -> a /'
#16	START S A B -> a	a\$	ERROR: Grammar error at line 2: expected exactly one uppercase nonterminal on the left side, found 'A B'
#17	START S S -> S A / a A -> A b S / b	a\$ab\$abb\$ abba\$aa\$b\$	ACCEPTED ACCEPTED ACCEPTED ACCEPTED REJECTED REJECTED
#18	START S S -> a	a\$	ACCEPTED

#19	START S S -> a	a\$\$	ERROR: Empty words are not supported.
#20	START S s -> a	a\$	ERROR: Grammar error at line 2: expected one uppercase nonterminal on the left side, found 's'

4.1 Detailed Expected Outputs For Accepted Cases

Case #1

Expected final result:

ACCEPTED

Expected reversed rightmost derivation:

(2)

Expected final parse tree:

FINAL PARSE TREE

```
S/2
└─ a
```

Case #2

Expected final result:

ACCEPTED

Expected reversed rightmost derivation:

(2,4,1)

Expected final parse tree:

FINAL PARSE TREE

```
S/1
├─ S/2
│   └─ a
└─ A/4
    └─ b
```

Case #3

Expected final result:

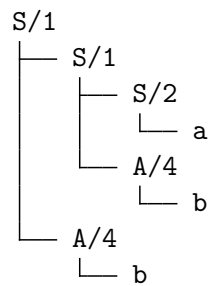
ACCEPTED

Expected reversed rightmost derivation:

(2,4,1,4,1)

Expected final parse tree:

FINAL PARSE TREE



Case #4

Expected final result:

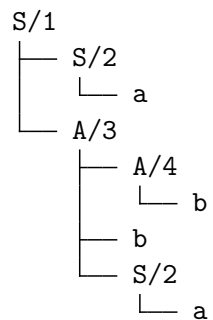
ACCEPTED

Expected reversed rightmost derivation:

(2,4,2,3,1)

Expected final parse tree:

FINAL PARSE TREE



Case #17

Expected final result:

ACCEPTED

ACCEPTED

ACCEPTED

ACCEPTED

REJECTED

REJECTED

Expected reversed rightmost derivation:

a\$:

(2)

ab\$:

(2,4,1)

abb\$:

(2,4,1,4,1)

abba\$:

(2,4,2,3,1)

Expected final parse tree:

FINAL PARSE TREE FOR a\$

S/2

└─ a

FINAL PARSE TREE FOR ab\$

S/1

└─ S/2
└─ A/4
 └─ b

FINAL PARSE TREE FOR abb\$

S/1

└─ S/1
└─ S/2
 └─ a
└─ A/4
 └─ b
└─ A/4
 └─ b

FINAL PARSE TREE FOR abba\$

S/1

└─ S/2
└─ A/3
 └─ A/4
 └─ b
 └─ b
 └─ S/2
 └─ a

Case #18

Expected final result:

ACCEPTED

Expected reversed rightmost derivation:

(1)

Expected final parse tree:

FINAL PARSE TREE

S/1
└─ a